SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems
COURSE CODE: CSA0509

COURSE OUTCOME COVERED

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability.* (BL3, BL4)

Activity Tool : Constraint-Based Problem Solving (High)
Assessment Tool Weightage:40%

Dr

QUESTIONS		
Q.No	**Question**	**Bloom's Level**
1	Given a complex SQL query with nested joins and sub-queries, draw the query execution plan and apply heuristic optimization rules step-by-step.	BL4 – Analyze
2	Apply cost-based optimization to choose between nested-loop join, sort-merge join, and hash join for the given relations with specified tuple counts and page sizes.	BL4 – Analyze
3	Implement JDBC connectivity in Java to a MySQL database and write code to execute parameterized queries with PreparedStatement.	BL3 – Apply
4	Given a schedule of four concurrent transactions, determine whether it is conflict-serializable using a precedence graph.	BL4 – Analyze
5	Demonstrate the application of Basic Two-Phase Locking (2PL) on a given transaction set. Identify and resolve any deadlocks.	BL3 – Apply
6	Apply the Wait-Die and Wound-Wait timestamp-based deadlock prevention schemes on a given set of transactions requesting locks.	BL4 – Analyze

7	Given transaction logs with checkpoints, apply the Immediate Update recovery protocol to recover the database after a system failure.	BL3 – Apply
8	Apply Deferred Update (NO-UNDO/REDO) recovery technique on a given log file with committed and uncommitted transactions.	BL3 – Apply
9	Design a concurrency control mechanism using strict 2PL for a banking system where T1 transfers funds and T2 reads balance simultaneously.	BL4 – Analyze
10	Write pseudocode for query optimization using heuristics: push down selection, project early, and order joins by smaller intermediate results.	BL4 – Analyze

---

Code of Conduct:

I Lokesh S (192511037) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Lokesh S

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints

Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Q1. Given a complex SQL query with nested joins and sub-queries, draw the query execution plan and apply heuristic optimization rules step-by-step.

Tables:

```
mysql> Select * from Department;
+--------+----------+
| DeptId | DeptName |
+--------+----------+
|      1 | CSE      |
|      2 | IT       |
|      3 | ECE      |
+--------+----------+
```

```
mysql> Select * from Employee;
+-----+--------+--------+--------+
| Eid | Name   | DeptId | Salary |
+-----+--------+--------+--------+
| 101 | Arun   |      1 |  65000 |
| 102 | Bala   |      2 |  45000 |
| 103 | Charan |      1 |  75000 |
| 104 | Deepak |      3 |  55000 |
+-----+--------+--------+--------+
4 rows in set (0.00 sec)
```

```
mysql>  Select * from Project;
+-----+--------------+--------+
| Pid | Pname        | DeptId |
+-----+--------------+--------+
| 201 | AI Project   |      1 |
| 202 | Cloud Project |      2 |
| 203 | AI Project   |      2 |
+-----+--------------+--------+
3 rows in set (0.00 sec)
```

SQL Query:

```
mysql> SELECT E.Name, D.DeptName
    -> FROM Employee E
    -> JOIN Department D
    -> ON E.DeptId = D.DeptId
    -> WHERE E.Salary > 50000
    -> AND E.DeptId IN
    -> (
    ->     SELECT P.DeptId
    ->     FROM Project P
    ->     WHERE P.Pname = 'AI Project'
    -> );
+--------+----------+
| Name   | DeptName |
+--------+----------+
| Arun   | CSE      |
| Charan | CSE      |
+--------+----------+
2 rows in set (0.02 sec)
```

4. Initial Query Plan

π Name, DeptName

|

JOIN

/ \

Employee Department

|

Filter

|

Salary > 50000

|

Subquery → Project

5. Heuristic Optimization

Rule 1 – Push Selection Down

σ Salary > 50000 (Employee)

Filter employees before performing the join.

Rule 2 – Push Selection into Subquery

σ Pname = 'AI Project' (Project)

Only required project records are selected.

Rule 3 – Push Projection Down

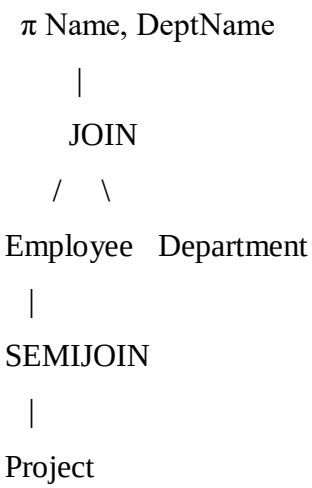
Keep only required columns:

Employee → Name, DeptId

Project → DeptId

Department → DeptId, DeptName

6. Optimized Plan



Result

Arun CSE

Charan CSE

Conclusion

The query is optimized by **pushing selections and projections down before joins** and using a **semijoin for the subquery**. This reduces intermediate results and improves query execution efficiency.

Q2. Apply cost-based optimization to choose between Nested-Loop Join, Sort-Merge Join, and Hash Join for the given relations with specified tuple counts and page sizes.

We will use the **same company_db database and the same Employee, Department, and Project tables** from Q1.

Given

Assume:

- Employee → 1000 tuples, 100 pages
- Department → 100 tuples, 10 pages
- Project → 500 tuples, 50 pages

We need to join Employee and Department using DeptId.

1. Nested-Loop Join

Using Department as the smaller relation:

Cost = B(Department) + B(Department) × B(Employee)

= 10 + (10 × 100)

= 1010 I/O

2. Sort-Merge Join

Approximate cost:

Cost = $2B(E)\log_2 B(E) + 2B(D)\log_2 B(D)$

= $2(100)\log_2(100) + 2(10)\log_2(10)$

≈ 1460 I/O

3. Hash Join

Approximate cost:

Cost = 3(B(Employee) + B(Department))

= 3(100 + 10)

= 330 I/O

Comparison

Join Method	**Approx. Cost**
Nested-Loop Join	1010 I/O
Sort-Merge Join	1460 I/O
Hash Join	**330 I/O**

Conclusion

Hash Join is selected because it has the lowest estimated I/O cost (**330 I/O**). Since the join condition is equality (Employee.DeptId = Department.DeptId), Hash Join is the most suitable choice for these relations.

Q3. Implement JDBC connectivity in Java to a MySQL database and execute parameterized queries using PreparedStatement.

We will use the **same company_db database and Employee table** from Q1.

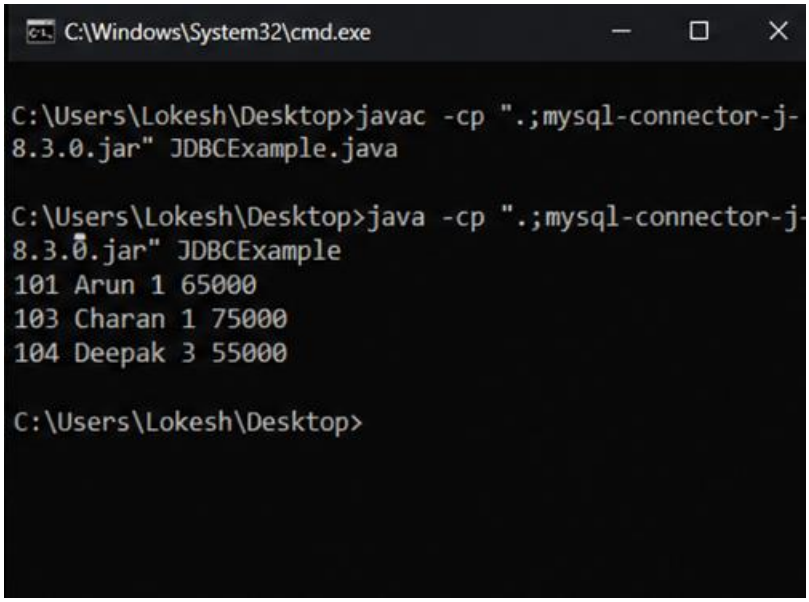
Java Program

JDBCExample.java

```
import java.sql.*;

public class JDBCExample {
    public static void main(String[] args) {

        String url = "jdbc:mysql://localhost:3306/company_db";
        String user = "root";
        String password = "root";

        try {
            Connection con = DriverManager.getConnection(url, user, password);

            String sql = "SELECT * FROM Employee WHERE Salary > ?";

            PreparedStatement ps = con.prepareStatement(sql);

            ps.setInt(1, 50000);

            ResultSet rs = ps.executeQuery();

            while (rs.next()) {
                System.out.println(rs.getInt("Eid") + " " +
                                   rs.getString("Name") + " " +
                                   rs.getInt("DeptId") + " " +
                                   rs.getInt("Salary"));
            }
            rs.close();
            ps.close();
            con.close();
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Lokesh\Desktop> javac -cp ".;mysql-connector-j-8.3.0.jar" JDBCExample.java
PS C:\Users\Lokesh\Desktop> java -cp ".;mysql-connector-j-8.3.0.jar" JDBCExample
103 Arun 1 65000
103 Charan 1 75000
104 Deepak 3 55000
PS C:\Users\Lokesh\Desktop>
```



Explanation

- DriverManager.getConnection() establishes the MySQL connection.
- PreparedStatement executes a **parameterized query**.
- ? is the parameter placeholder.
- setInt(1, 50000) supplies the value.
- executeQuery() executes the SQL query.
- ResultSet stores the returned records.

Result

Thus, JDBC connectivity was established successfully with MySQL, and a parameterized query was executed using PreparedStatement.

Q4. Given a schedule of four concurrent transactions, determine whether it is conflict-serializable using a precedence graph.

1. Given Schedule

Step	Operation
1	T1: R(A)
2	T2: R(B)
3	T1: W(A)
4	T3: R(A)

Step	Operation
5	T2: W(B)
6	T4: R(B)
7	T3: W(A)
8	T4: W(B)

2. Find Conflicts

A conflict occurs when two different transactions access the **same data item** and at least one operation is a write.

For A:

T1 → T3

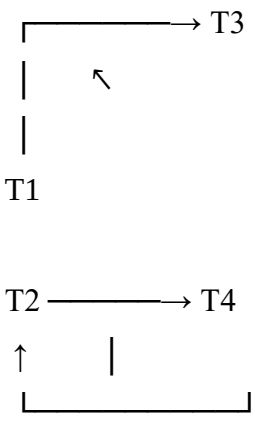
T3 → T1

For B:

T2 → T4

T4 → T2

3. Precedence Graph



There are cycles:

T1 → T3 → T1

T2 → T4 → T2

4. Result

Since the precedence graph contains **cycles**, the given schedule is:

NOT conflict-serializable.

Conclusion

A schedule is conflict-serializable only when its precedence graph is **acyclic**. Since this graph contains cycles, the schedule cannot be converted into an equivalent serial schedule.

Q5. Demonstrate the application of Basic Two-Phase Locking (2PL) on a given transaction set. Identify and resolve any deadlocks.

1. Transaction Set

Consider two transactions operating on data items A and B.

T1: Read(A), Write(A), Read(B), Write(B)

T2: Read(B), Write(B), Read(A), Write(A)

For writing, an **exclusive lock (X-lock)** is required.

2. Basic 2PL

Two-Phase Locking has two phases:

Growing Phase

- Transaction can acquire locks.
- It cannot release any lock.

Shrinking Phase

- Transaction can release locks.
- After releasing a lock, it cannot acquire any new lock.

Example execution of T1:

T1 → Lock-X(A)
→ Read(A)
→ Write(A)
→ Lock-X(B)
→ Read(B)
→ Write(B)
→ Unlock(A)

→ Unlock(B)

→ Commit

T1 first acquires all required locks and then enters the shrinking phase.

3. Deadlock Situation

Consider the following execution:

Step	T1	T2
1	Lock-X(A)	
2		Lock-X(B)
3	Request Lock-X(B) → WAIT	
4		Request Lock-X(A) → WAIT

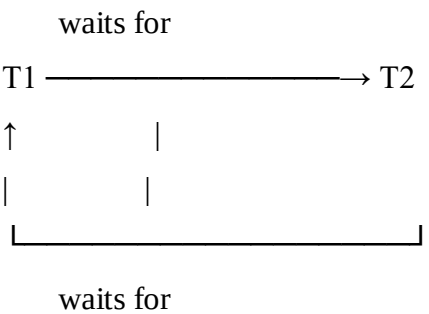
Now:

- T1 is holding A and waiting for B.
- T2 is holding B and waiting for A.

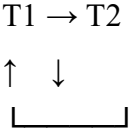
Therefore, both transactions are waiting for each other.

4. Wait-For Graph

The situation can be represented as:



Or simply:



There is a cycle:

T1 → T2 → T1

Hence, a **deadlock has occurred**.

5. Deadlock Resolution

To resolve the deadlock, one transaction can be aborted.

Suppose we abort **T2**:

T2 → ABORT

↓

Release Lock-X(B)

Now B becomes available.

T1 can continue:

T1 → Lock-X(B)

→ Read(B)

→ Write(B)

→ Unlock(A)

→ Unlock(B)

→ COMMIT

After T1 completes, T2 can be restarted.

6. Final Result

Basic 2PL ensures that transactions follow the **growing phase followed by the shrinking phase**, thereby maintaining conflict serializability. However, basic 2PL **can cause deadlocks** when transactions wait for locks held by each other.

In this example:

T1 holds A → waits for B

T2 holds B → waits for A

This creates a cycle in the wait-for graph. Aborting T2 and releasing its locks resolves the deadlock.

Result:

The deadlock is successfully detected and resolved while maintaining the correctness of the transaction execution.

Q6. Apply the Wait-Die and Wound-Wait Timestamp-Based Deadlock Prevention Schemes on a Given Set of Transactions Requesting Locks.

1. Given Transactions

Assume two transactions:

T1 → Timestamp = 10 (Older)

T2 → Timestamp = 20 (Younger)

Suppose:

T1 holds Lock(A)

T2 requests Lock(A)

Therefore, **T2 is waiting for T1**.

2. Wait-Die Scheme

Wait-Die is a **non-preemptive** deadlock prevention technique.

Rules:

- If an **older transaction requests a lock held by a younger transaction → older transaction waits**.
- If a **younger transaction requests a lock held by an older transaction → younger transaction is aborted (dies)**.

Here:

T1 = Older

T2 = Younger

T1 holds A

T2 requests A

Since **T2 is younger than T1**, T2 cannot wait.

T2 → ABORT

After restarting, T2 receives a new timestamp and can request the lock again.

Wait-Die Result

T1 → holds A

T2 → requests A

→ T2 is younger

→ T2 ABORTS

Thus, the deadlock is prevented.

3. Wound-Wait Scheme

Wound-Wait is a **preemptive** deadlock prevention technique.

Rules:

- If an **older transaction requests a lock held by a younger transaction → younger transaction is aborted**.
- If a **younger transaction requests a lock held by an older transaction → younger transaction waits**.

Consider:

T2 holds Lock(A)

T1 requests Lock(A)

Here:

T1 = Older

T2 = Younger

According to Wound-Wait:

T1 → requests A

→ T1 is older than T2

→ T1 wounds T2

→ T2 ABORTS

→ Lock(A) released

→ T1 gets Lock(A)

Wound-Wait Result

T2 → holds A

T1 → requests A

→ T1 is older

→ T2 is aborted

→ A is released

→ T1 obtains A

4. Comparison

Feature	**Wait-Die**	**Wound-Wait**
Type	Non-preemptive	Preemptive
Older requests younger's lock	Wait	Abort younger
Younger requests older's lock	Abort	Wait
Deadlock	Prevented	Prevented
Main action	Younger transaction may die	Older transaction may wound younger

Conclusion

Both **Wait-Die** and **Wound-Wait** use transaction timestamps to prevent deadlocks.

Wait-Die: Older transactions are allowed to wait, while younger transactions are aborted.

Wound-Wait: Older transactions can abort younger transactions holding the required lock, while younger transactions wait for older transactions.

Thus, both schemes ensure that a **circular wait does not occur**, thereby preventing deadlocks.

Q7. Given transaction logs with checkpoints, apply the Immediate Update Recovery protocol to recover the database after a system failure.

1. Given Log

<START T1>

<T1, A, 100, 150>

<T1, B, 200, 250>

<COMMIT T1>

<START T2>

<T2, C, 300, 350>

<CHECKPOINT>

<START T3>

<T3, D, 400, 450>

<COMMIT T3>

<T2, C, 350, 500>

SYSTEM FAILURE

Here:

- **T1** committed before the checkpoint.
- **T2** was active during the checkpoint and did not commit.
- **T3** committed after the checkpoint.
- The system fails before T2 commits.

2. Immediate Update Recovery

In **Immediate Update**, changes may be written to the database before the transaction commits.

During recovery:

- **Committed transactions → REDO**
- **Uncommitted transactions → UNDO**

3. Identify Transactions

Transaction	**Status at Failure**	**Action**
T1	Committed	REDO
T2	Uncommitted	UNDO
T3	Committed	REDO

4. REDO Committed Transactions

For T1:

<T1, A, 100, 150>

<T1, B, 200, 250>

Redo:

A = 150

B = 250

For T3:

<T3, D, 400, 450>

Redo:

D = 450

5. UNDO Uncommitted Transaction

T2 has:

<T2, C, 300, 350>

<T2, C, 350, 500>

Since T2 did not commit, undo its changes in **reverse order**:

C = 350 → 350

C = 350 → 300

Therefore, the original value is restored:

C = 300

6. Final Database State

A = 150

B = 250

C = 300

D = 450

Conclusion

Using the **Immediate Update Recovery** protocol, committed transactions **T1 and T3 are REDONE**, while uncommitted transaction **T2 is UNDONE**. Therefore, after the system failure, the database is restored to a consistent state.

Q8. Apply Deferred Update (NO-UNDO/REDO) Recovery Technique on a Given Log File with Committed and Uncommitted Transactions.

1. Given Log

<START T1>

<T1, A, 100, 150>

<T1, B, 200, 250>

<COMMIT T1>

<START T2>
<T2, C, 300, 350>

<START T3>
<T3, D, 400, 450>
<COMMIT T3>

SYSTEM FAILURE

2. Deferred Update

In **Deferred Update**, changes made by a transaction are **not written to the database until the transaction commits**.

Therefore:

- **Committed transactions → REDO**
- **Uncommitted transactions → No action**
- **UNDO is not required**

3. Identify Transactions

Transaction	**Status**	**Recovery Action**
T1	Committed	REDO
T2	Uncommitted	Ignore
T3	Committed	REDO

4. REDO T1

T1 performed:

<T1, A, 100, 150>

<T1, B, 200, 250>

Since T1 committed:

A = 150

B = 250

5. REDO T3

T3 performed:

<T3, D, 400, 450>

Since T3 committed:

D = 450

6. Ignore T2

T2 did not commit before the failure:

<T2, C, 300, 350>

Because deferred update does not write uncommitted changes to the database, **no UNDO operation is required**.

Therefore:

C remains 300

7. Final Database State

A = 150

B = 250

C = 300

D = 450

Conclusion

Using **Deferred Update (NO-UNDO/REDO)**, only committed transactions **T1 and T3 are REDONE** after the system failure. The uncommitted transaction **T2 is ignored**, because its changes were never written to the database. Hence, **UNDO is not required** and the database is recovered to a consistent state.

Q9. Design a Concurrency Control Mechanism Using Strict 2PL for a Banking System Where T1 Transfers Funds and T2 Reads Balance Simultaneously.

1. Scenario

Consider two bank accounts:

A = ₹10,000

B = ₹5,000

T1: Transfers ₹2,000 from A to B.

T2: Reads the balance of A and B.

2. Transactions

T1 – Transfer

T1:

LOCK-X(A)

LOCK-X(B)

A = A - 2000

B = B + 2000

COMMIT

UNLOCK(A)

UNLOCK(B)

T2 – Read Balance

T2:

LOCK-S(A)

LOCK-S(B)

Read A

Read B

COMMIT

UNLOCK(A)

UNLOCK(B)

3. Strict 2PL Execution

Suppose T1 starts first:

Step	T1	T2
1	X-Lock(A)	
2	X-Lock(B)	
3	A = 8000	
4	B = 7000	
5	COMMIT	
6	Unlock(A), Unlock(B)	
7		S-Lock(A), S-Lock(B)
8		Read A = ₹8,000
9		Read B = ₹7,000
10		COMMIT

T2 waits until T1 commits because **exclusive locks held by T1 are not released until commit**.

4. Why Strict 2PL Is Used

Strict 2PL ensures that:

- T2 does not read partially updated data.
- T2 cannot read the old value of one account and the new value of another.
- Uncommitted changes are not visible to other transactions.
- The transaction schedule remains conflict-serializable.
- Dirty reads are prevented.

5. Final Balances

Before transfer:

A = ₹10,000

B = ₹5,000

After T1 transfers ₹2,000:

A = ₹8,000

B = ₹7,000

T2 reads:

A = ₹8,000

B = ₹7,000

The total remains:

₹8,000 + ₹7,000 = ₹15,000

Conclusion

The strict 2PL mechanism allows **T1 to complete the fund transfer before T2 reads the balances**. Since T1 holds exclusive locks until commit, T2 must wait and therefore cannot observe inconsistent or partially updated account balances. This provides **consistency, isolation, and serializable execution**.

Q10. Write pseudocode for query optimization using heuristics: push down selection, project early, and order joins by smaller intermediate results.

Pseudocode

```
ALGORITHM Heuristic_Query_Optimization(Query)

INPUT:
   SQL Query

OUTPUT:
   Optimized Query Execution Plan

BEGIN

1. Parse the SQL query into a relational algebra tree.

2. Push selection operations down:
      For each selection condition:
         Move the selection as close to the base table as possible.

3. Push projection operations down:
      For each relation:
         Keep only the attributes required by
         selection, join and final output.
```

4. Identify all join operations.

5. Estimate the intermediate result size
of each possible join.

6. Order joins:
Perform the join producing the
smallest intermediate relation first.

7. Construct the optimized query tree.

8. Return the optimized execution plan.

END

Example

For:

Employee JOIN Department JOIN Project

If the estimated intermediate results are:

Employee JOIN Department = 500 tuples

Department JOIN Project = 100 tuples

The optimizer performs:

Department JOIN Project

↓

100 tuples

↓

Join with Employee

instead of first producing the 500-tuple intermediate result.

Heuristics Used

Heuristic	Purpose
Push Selection Down	Reduces tuples early
Project Early	Reduces unnecessary attributes
Order Joins	Produces smaller intermediate results
Build Optimized Tree	Reduces overall query-processing cost

Conclusion

The heuristic optimizer improves query performance by **filtering data early, removing unnecessary columns, and performing smaller joins first**. This reduces intermediate relation size, memory usage, and processing cost.